

JavaScript Function Arguments

[Back to JS page](#)

Table of Contents

- [JavaScript Function Arguments](#)
 - [Parameters vs. Arguments](#)
 - [Example 1](#)
 - [The Arguments Object](#)
 - [Example 2](#)
 - [The Order of Arguments Matters](#)
 - [Example 3](#)
 - [Arguments Can Be Variables](#)
 - [Example 4](#)
 - [Missing Arguments](#)
 - [Example 5](#)
 - [Default Parameters](#)
 - [Example 6](#)
 - [Function Rest Parameter](#)
 - [Example 7](#)
 - [Document](#)
 - [Reference](#)

Parameters vs. Arguments

Function **parameters** are the names listed in the function definition.

Function **arguments** are the real values passed to (and received by) the function.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Parameters vs. Arguments</h4>
<p id="demo"></p>

<script>
function myFunction(firstName, lastName) {
  return "Hello " + firstName + " " + lastName;
}
// "John" and "Doe" are arguments
document.getElementById("demo").innerHTML = myFunction("John", "Doe");
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The Arguments Object

JavaScript functions have a built-in object called the `arguments` object.

The `arguments` object contains an array of the arguments used when the function was called:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Arguments Object</h4>
<p id="demo"></p>

<script>
function findMax() {
  let max = -Infinity;
  for (let i = 0; i < arguments.length; i++) {
    if (arguments[i] > max) {
      max = arguments[i];
    }
  }
  return max;
}
document.getElementById("demo").innerHTML = "Max of (4, 7, 2, 9, 5): " + findMax(4, 7, 2, 9, 5);
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

The Order of Arguments Matters

The order of arguments passed to a function must match the order of parameters defined:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Order of Arguments Matters</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function carInfo(brand, model, year) {
    return brand + " " + model + " (" + year + ")";
}
// Correct order
document.getElementById("demo1").innerHTML = "Correct: " + carInfo("Toyota", "Corolla", 2020);
// Wrong order – produces different result
document.getElementById("demo2").innerHTML = "Wrong: " + carInfo(2020, "Toyota", "Corolla");
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Arguments Can Be Variables

Arguments can be variables, not just literal values:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Arguments Can Be Variables</h4>
<p id="demo"></p>

<script>
function add(x, y) {
    return x + y;
}
let a = 10;
let b = 25;
document.getElementById("demo").innerHTML = a + " + " + b + " = " + add(a, b);
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Missing Arguments

If a function is called with missing arguments (fewer than declared), the missing values are set to `undefined` :

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Missing Arguments</h4>
<p id="demo"></p>

<script>
function multiply(a, b) {
  return a * b;
}
document.getElementById("demo").innerHTML = "multiply(5) = " + multiply(5) + " (NaN because b is
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Default Parameters

ES6 allows function parameters to have default values, used when arguments are missing:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Default Parameters</h4>
<p id="demo1"></p>
<p id="demo2"></p>

<script>
function myFunction(x, y = 10) {
  return x + y;
}
document.getElementById("demo1").innerHTML = "myFunction(5) = " + myFunction(5);
document.getElementById("demo2").innerHTML = "myFunction(5, 3) = " + myFunction(5, 3);
</script>

</body>
</html>
```

Example 6

Result [View Example](#)

Function Rest Parameter

The rest parameter (`...`) allows a function to treat an indefinite number of arguments as an array:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Function Arguments</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Rest Parameter</h4>
<p id="demo"></p>

<script>
function sum(...args) {
  let total = 0;
  for (let arg of args) {
    total += arg;
  }
  return total;
}
document.getElementById("demo").innerHTML = "Sum of (1, 2, 3, 4, 5): " + sum(1, 2, 3, 4, 5);
</script>

</body>
</html>
```

Example 7

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Function Arguments](#)